COURSE PROJECT REPORT: OBJECT ORIENTED PROGRAMMING IN C++

1. PROJECT TITLE

Project Name : Restaurant Order & Billing System
Project Code : CPP07
Enrollment : IU2541230417
Academic Term: 2026-2027

2. STUDENT DETAILS

Student Name : Nabhit Shah
Enrollment Number: IU2541230417
Project Code : CPP07
Assigned Domain : Restaurant Order & Billing System
Submission Date : October 11, 2026

3. PROBLEM STATEMENT

Traditional food service establishments frequently struggle with manual paper-based order taking and billing. Common pain points include:
1. Calculation errors in subtotals, tax computation, and discount deductions.
2. Inefficient communication between dining staff and inventory/cashier counters.
3. Lack of accurate historical records for sales analysis and popular menu tracking.
4. Vulnerability to data loss and billing discrepancies during peak operating hours.

To resolve these challenges, the Restaurant Order & Billing System provides a robust, modular, console-based desktop application written in C++17. The system automates menu catalog maintenance, customer directory management, multi-item order placement, automatic GST taxation and discounts, receipt generation, and persistent disk storage.

4. OBJECTIVES

- Develop a modular, object-oriented software system in standard C++17.
- Implement strict encapsulation with private data members and public interfaces.
- Utilize inheritance and runtime polymorphism for specialized menu items.
- Implement dynamic memory allocation (new[] and delete[]) adhering to the Rule of Three.
- Implement reliable file handling to ensure complete data persistence across sessions.
- Provide a minimum 9-option interactive main menu with input validation and exception handling.
- Generate managerial reports summarizing revenue, taxes, discounts, and item tallies.

5. FEATURES AND MENU OPERATIONS

The system features a structured menu-driven workflow:

Option 1: Add Record
  - Add specialized Food Items (with cuisine type and vegetarian flag).
  - Add specialized Beverage Items (with serving size and hot/cold indicator).
  - Register new Customers with mandatory 10-digit phone number validation.

Option 2: Display Records
  - Tabular polymorphic display of menu items showing ID, Name, Price, Category,
    Type-specific attributes, and Current Availability.
  - Formatted display of customer records, active/past orders, and generated bil
ls.

Option 3: Search Record
  - Search menu items, customers, orders, and bills by ID or case-insensitive ke
yword.

Option 4: Update Record
  - Modify item price, category, name, or availability.
  - Modify customer contact details.
  - Update order workflow status (Pending, Completed, Cancelled).

Option 5: Delete Record
  - Delete menu items or customers with user confirmation and dynamic array real
ignment.

Option 6: Place Order (Main Transaction)
  - Interactive multi-step order placement:
    Step 1: Select existing customer or register new customer on-the-fly.
    Step 2: Display polymorphic menu items.
    Step 3: Dynamically add items and quantities to the Order object.
    Step 4: Display order summary and calculate subtotal.
    Step 5: Apply GST tax rate (default 18%) and flat discounts.
    Step 6: Instantiate Bill, calculate final total, and display formatted recei
pt.
    Step 7: Collect payment (Cash, Card, UPI), mark bill as Paid, and complete o
rder.

Option 7: Generate Report
  - Displays system-wide counts, pending vs completed orders, total revenue,
    total GST collected, total discounts given, and top 3 popular ordered items.

Option 8: Save/Load Data
  - Manual on-demand persistence synchronization.

Option 9: Exit
  - Automatically saves all modified records to text files and safely terminates
.

6. CLASSES AND RESPONSIBILITIES

The system is built upon 8 well-defined classes across modular .h/.cpp files:

1. MenuItem (MenuItem.h / MenuItem.cpp) [Base Class]
   - Responsibility: Abstract base class representing generic catalog items.
   - Data Members:
     * int itemId
     * string name
     * double price
     * string category
     * bool available

- Member Functions:
  * Constructors (default & parameterized), virtual destructor ~MenuItem()
  * Getters and setters for all attributes
  * virtual void display() const (pure/runtime polymorphic display)
  * virtual string getType() const (returns "FOOD", "BEVERAGE", "GENERIC")
  * virtual string toFileString() const (serialization)
  * virtual void fromFileString(const string& line) (deserialization)

2. FoodItem (FoodItem.h / FoodItem.cpp) [Derived Class]
- Responsibility: Represents edible items with culinary attributes.
- Data Members:
  * string cuisineType (e.g., North Indian, Chinese, Italian)
  * bool isVeg (Vegetarian flag)
- Member Functions:
  * Constructors, destructor
  * Overrides display(), getType(), toFileString(), fromFileString()

3. BeverageItem (BeverageItem.h / BeverageItem.cpp) [Derived Class]
- Responsibility: Represents drinks with serving size and temperature.
- Data Members:
  * string size (Small, Medium, Large)
  * bool isHot (Hot vs Cold)
- Member Functions:
  * Constructors, destructor
  * Overrides display(), getType(), toFileString(), fromFileString()

4. Customer (Customer.h / Customer.cpp)
- Responsibility: Manages diner identification and contact details.
- Data Members:
  * int customerId, string name, string phone, string address
- Member Functions:
  * Constructors, destructor, getters/setters, display(), toFileString(), fro
mFileString()

5. Order (Order.h / Order.cpp) [Dynamic Memory Management]
- Responsibility: Represents a dining transaction containing dynamic item arr
ays.
- Data Members:
  * int orderId, int customerId
  * int* itemIds (dynamically allocated array: new int[capacity])
  * int* quantities (dynamically allocated array: new int[capacity])
  * int itemCount, int capacity
  * string status (Pending, Completed, Cancelled), string orderDate
- Member Functions:
  * Order(), Order(orderId, customerId)
  * Order(const Order& other) [Deep Copy Constructor - Rule of Three]
  * Order& operator=(const Order& other) [Copy Assignment - Rule of Three]
  * ~Order() [Destructor freeing itemIds and quantities via delete[]]
  * void resize() [Private helper: doubles array capacity when full]
  * void addItem(int itemId, int quantity)
  * void removeItem(int itemId)
  * display(), toFileString(), fromFileString()

6. Bill (Bill.h / Bill.cpp)
- Responsibility: Manages taxation, discounts, receipts, and payment status.
- Data Members:
  * int billId, int orderId, int customerId
  * double subtotal, double taxRate, double taxAmount, double discount, doubl
e totalAmount
  * string paymentStatus (Unpaid, Paid), string paymentMethod, string billDat
e
- Member Functions:

```
     * Constructors, destructor, getters/setters
     * void calculateTotal(): Computes taxAmount and totalAmount = subtotal + ta
x - discount
     * void display(): Renders formatted receipt
     * toFileString(), fromFileString()

7. FileManager (FileManager.h / FileManager.cpp)
   - Responsibility: Encapsulates file persistence routines using C++ fstream.
   - Member Functions:
     * void saveMenuItems(MenuItem** items, int count, const string& file) const
     * void saveCustomers(Customer* customers, int count, const string& file) co
nst
     * void saveOrders(Order* orders, int count, const string& file) const
     * void saveBills(Bill* bills, int count, const string& file) const
     * int loadMenuItems(MenuItem**& items, int& capacity, const string& file) c
onst
     * int loadCustomers(Customer*& customers, int& capacity, const string& file
) const
     * int loadOrders(Order*& orders, int& capacity, const string& file) const
     * int loadBills(Bill*& bills, int& capacity, const string& file) const

8. Restaurant (Restaurant.h / Restaurant.cpp) [System Controller]
   - Responsibility: Coordinates the overall lifecycle, dynamic collections, use
r input,
     main transaction execution, reports, and exception handling.
   - Data Members:
     * MenuItem** menuItems (Dynamic array of base pointers for polymorphism)
     * Customer* customers (Dynamic array)
     * Order* orders (Dynamic array)
     * Bill* bills (Dynamic array)
     * int menuCount, customerCount, orderCount, billCount
     * int menuCapacity, customerCapacity, orderCapacity, billCapacity
     * FileManager fileManager
     * string dataPath
   - Member Functions:
     * run(), addMenuItem(), displayMenuItems(), searchMenuItem(), updateMenuIte
m(),
       deleteMenuItem(), addCustomer(), displayCustomers(), placeOrder(), genera
teReport(),
       saveData(), loadData()


7. OOP CONCEPTS DEMONSTRATED WITH CODE EXAMPLES
--------------------------------------------------------------------------------
1. Classes and Objects:
   Each domain entity is modeled as a class. Real-world restaurant components
   (Food, Beverage, Customer, Order, Bill) are instantiated as objects during ru
ntime.

2. Encapsulation:
   All attributes are declared private or protected. External access is mediated
   exclusively via public member functions and getters/setters.
   Example:
     class Customer {
     private:
         int customerId;
         string name, phone, address;
     public:
         string getName() const { return name; }
         void setName(const string& n) { name = n; }
     };
```

3. Constructors and Destructors:
   Default and parameterized constructors are implemented across all 8 classes.
   Destructors systematically release resources.
   Example:
```
     Order::Order(int orderId, int customerId) : orderId(orderId), customerId(cu
stomerId),
         itemCount(0), capacity(5), status("Pending") {
         itemIds = new int[capacity];
         quantities = new int[capacity];
     }
     Order::~Order() {
         delete[] itemIds;
         delete[] quantities;
     }
```

4. Inheritance:
   FoodItem and BeverageItem extend MenuItem, inheriting core properties while
   adding specialized domain fields (cuisineType, isVeg, size, isHot).
   Example:
```
     class FoodItem : public MenuItem {
     private:
         string cuisineType;
         bool isVeg;
     public:
         FoodItem(int id, string n, double p, string c, string cuisine, bool veg
)
             : MenuItem(id, n, p, c), cuisineType(cuisine), isVeg(veg) {}
     };
```

5. Runtime Polymorphism:
   The Restaurant class stores pointers of base type MenuItem*:
```
     MenuItem** menuItems = new MenuItem*[capacity];
```
   When iterating, virtual methods dispatch dynamically at runtime based on the
actual
   derived object type:
```
     menuItems[i]->display(); // Dispatches to FoodItem::display() or BeverageIt
em::display()
```

6. Dynamic Memory Management & Rule of Three:
   The Order class maintains dynamic arrays allocated on the free store with new
[].
   To prevent double-deletion and pointer aliasing, the Rule of Three is strictl
y implemented:
     - Destructor: frees allocated memory
     - Copy Constructor: performs deep copying
     - Copy Assignment Operator: deallocates old buffers and deep-copies element
s

8. FILE HANDLING AND DATA PERSISTENCE

Data is persisted in the Data/ directory using standard pipe-delimited text file
s:
- Data/menu_items.txt : Stores FoodItem and BeverageItem records.
- Data/customers.txt : Stores registered diner records.
- Data/orders.txt   : Stores orders and comma-separated item lists (id:qty).
- Data/bills.txt   : Stores billing receipts, taxes, discounts, and payment m
ethods.

Persistence Workflow:
1. Application Start: loadData() invokes FileManager to parse files and reconstr
uct

   in-memory dynamic arrays.
2. In-Memory Processing: All additions, edits, deletions, and transactions mutat
e dynamic arrays.
3. Application Exit: saveData() serializes all records back to disk, preserving
changes.
Verification: The application was tested by adding records, closing the binary,
and
re-launching. All newly created records were accurately reloaded.

9. UML DIAGRAMS

The project contains 4 UML diagrams placed in the Diagrams/ directory:
1. UseCase_Diagram.svg / .mermaid : Use Case diagram featuring Customer and Mana
ger actors
   interacting with 8 core use cases with <<include>> dependencies.
2. Class_Diagram.svg / .mermaid   : Comprehensive class diagram detailing all 8
classes,
   visibility qualifiers (#, -, +), data types, inheritance, and aggregations.
3. Sequence_Diagram_1.svg / .mermaid : Sequence diagram for the Main Transaction
 (Place Order),
   tracing method calls across User, Restaurant, Customer, MenuItem, Order, and
Bill.
4. Sequence_Diagram_2.svg / .mermaid : Sequence diagram for Data Persistence, sh
owing startup
   loading and shutdown saving via FileManager and file streams.

10. PROGRAM WORKFLOW
---
1. Entry: main() instantiates Restaurant restaurant; and calls restaurant.run();
2. Startup: Restaurant invokes loadData(), populating dynamic arrays from disk.
3. Menu Loop: Displays the 9-option menu, accepts validated integer choice.
4. Branching: Dispatches to sub-menus (Add, Display, Search, Update, Delete),
   Main Transaction (Place Order), Report, or Save/Load.
5. Transaction: Places order, calculates billing with tax/discount, collects pay
ment.
6. Termination: User selects option 9; saveData() flushes arrays to disk, destru
ctors
   cleanly release all heap memory, and program exits safely with return code 0.

11. TESTING MATRIX

TC01: Startup Data Loading
  Input   : Run executable ./Executable/Project
  Expected: Loads existing records from Data/*.txt and displays count banner.
  Actual : "Successfully loaded 10 menu items, 4 customers, 4 orders, 3 bills."
 -> PASS

TC02: Add Food Item
  Input   : Option 1 -> 1 -> Name: Paneer Tikka, Price: 240, Cat: Starters, Cuis
ine: North Indian, Veg: 1
  Expected: Assigns auto-increment ID and appends to menuItems array.
  Actual : "Menu Item added successfully with ID: 11" -> PASS

TC03: Phone Number Validation
  Input   : Option 1 -> 2 -> Name: Test, Phone: 12345 (5 digits)
  Expected: Error message "Phone number must be exactly 10 digits!" and rejectio
n.
  Actual : Error displayed, record rejected -> PASS

TC04: Polymorphic Display
  Input   : Option 2 -> 1 (Menu Items)
  Expected: Displays Food items with cuisine/veg and Beverage items with size/te
mperature.
  Actual : Correct polymorphic table formatting displayed -> PASS

TC05: Search Menu Item
  Input   : Option 3 -> 1 -> Option 2 (Name) -> "Biryani"
  Expected: Finds and prints matching item "Chicken Biryani".
  Actual : Exact match displayed with details -> PASS

TC06: Place Order (Main Transaction)
  Input   : Option 6 -> Select Cust 1 -> Add Item 1 (Qty 2) -> Add Item 6 (Qty 2
) -> 18% GST -> Rs.50 Disc -> UPI
  Expected: Order created, subtotal calculated, tax Rs.169.20, disc Rs.50, Bill
generated, status Paid.
  Actual : Complete bill rendered, Order marked Completed, Bill marked Paid ->
PASS

TC07: Statistical Report
  Input   : Option 7
  Expected: Accurate aggregation of total orders, revenue, tax, discounts, top 3
 items.
  Actual : Revenue Rs.2396.20, Tax Rs.376.20, Top items accurately ranked -> PA
SS

TC08: Update Record
  Input   : Option 4 -> 1 -> ID 4 -> Option 2 (Price) -> 350
  Expected: Price of Item 4 updated to Rs.350.00.
  Actual : Price updated successfully -> PASS

TC09: Delete Record
  Input   : Option 5 -> 1 -> ID 11 -> Confirm 'y'
  Expected: Item 11 deleted, heap memory freed, array shifted left.
  Actual : "Menu Item deleted successfully!" -> PASS

TC10: Persistence Verification
  Input   : Modify data -> Exit (Option 9) -> Re-open executable
  Expected: Modified data remains persistent on disk and loads back accurately.
  Actual : Data verified persistent across restarts -> PASS

12. CHALLENGES AND SOLUTIONS

Challenge 1: Object Slicing with Polymorphism
- Issue: Storing derived objects in value-type arrays slices off derived fields.
- Solution: Used an array of base class pointers (MenuItem**) in Restaurant and
dynamically
  instantiated FoodItem or BeverageItem using new.

Challenge 2: Rule of Three Memory Management
- Issue: Default shallow copying of Order objects caused double-free errors on d
estruction.
- Solution: Explicitly implemented a deep-copy constructor, copy assignment oper
ator,
  and destructor to manage the dynamically allocated itemIds and quantities arra
ys.

Challenge 3: C++ Stream Extraction and Buffer Overflow
- Issue: cin >> left trailing newline characters, causing subsequent getline() c
alls to fail.
- Solution: Used cin.ignore(numeric_limits<streamsize>::max(), '

') and cin.clear() across
  all input paths to ensure robust stream state recovery.

Challenge 4: Multi-Item Serialization
- Issue: Representing dynamic arrays of items and quantities inside a flat single-line record.
- Solution: Implemented nested string tokenization using stringstream, formatting item pairs
  as id1:qty1,id2:qty2 separated by commas within the pipe-delimited line.

13. CONCLUSION

The Restaurant Order & Billing System successfully fulfills all academic and functional
specifications outlined in the project blueprint:
- Demonstrates thorough mastery of Object-Oriented Programming principles in C++17.
- Proves practical application of inheritance, runtime polymorphism, and dynamic arrays.
- Implements robust data persistence, defensive input validation, and statistical analysis.
- The project structure is clean, modular, fully documented, and ready for submission.

==========

END OF PROJECT REPORT

==========